

Outlines

- Introduction to Class and Objects
- Initializing an Object
- Access Specifiers/Modifiers
- Defining a Class with Member Functions
- Private Member Functions
- Static Data Members
- Static Member Functions

Introduction to Class and Objects

What is a Class?

- A blueprint for creating objects.
- Contains data members (attributes) and member functions (methods).

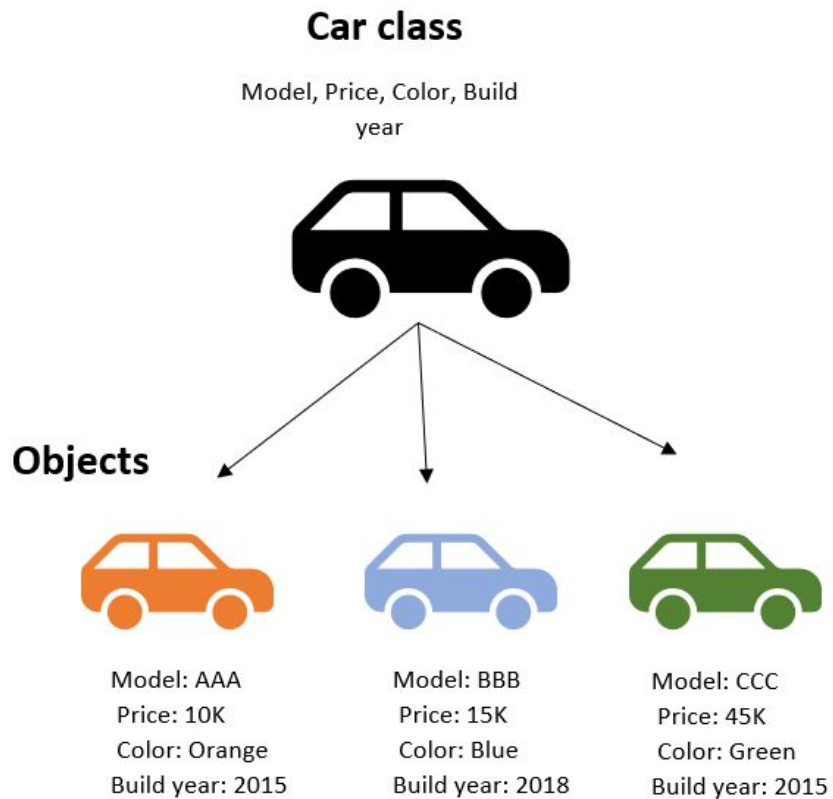
Components:

- **Attributes (Data Members):** Variables to hold data.
- **Methods (Member Functions):** Functions to manipulate the data.

Definition of Object:

- An instance of a class.
- Represents a real-world entity with state and behavior.
- Each object has its own copy of the data members defined in the class and can use the member functions to interact with these data members.

Example



Class Name : Car

Attributes:

- Model
- Price
- Color
- Build Year

Methods:

- drive()
- brake()

Objects:

- car1
- car2
- car3

Syntax of Class

A class in C++ is defined using the **class** keyword.

```
class ClassName {  
public:  
    // Public members  
private:  
    // Private members  
};
```

Syntax of Object

```
ClassName objectName;
```

Defining a Simple Class

```
class Car {  
public:  
    string brand;  
    string model;  
    int year;  
  
    void displayDetails () {  
        cout << "Brand: " << brand << endl;  
        cout << "Model: " << model << endl;  
        cout << "Year: " << year << endl;  
    }  
};
```

Creating Objects

```
Car myCar;  
myCar.brand = "Toyota";  
myCar.model = "Corolla";  
myCar.year = 2020;  
myCar.displayDetails();
```

Q. Write a program that defines a class named Car. The class should have public data members to store the car's brand, model, and year. It should also have a member function to display these details.

```
#include <iostream>
using namespace std;
```

```
class Car {
public:
    string brand;
    string model;
    int year;

    void displayDetails() {
        cout << "Brand: " << brand << endl;
        cout << "Model: " << model << endl;
        cout << "Year: " << year << endl;
    }
};
```

```
int main() {
    Car myCar;
    myCar.brand = "Toyota";
    myCar.model = "Corolla";
    myCar.year = 2020;
    myCar.displayDetails();

    return 0;
}
```

Try it!!

Q1. Create a class named 'Student' with a string variable 'name' and an integer variable 'roll_no'. Assign the value of roll_no and name by creating an object of the class Student and print those values.

Access Specifiers

Access specifiers in C++ are keywords that define how class members (variables and functions) can be accessed.

They are used to protect data and define what parts of your code can use certain parts of a class.

A class can have multiple public, protected, or private labeled sections. Each section remains in effect until either another section label or the closing right brace of the class body is seen.

The default access for members and classes is private.

Types:

- Public
- Private
- Protected

Public

Definition: Members declared as **public** can be accessed from anywhere in the program where the object of the class is visible.

Use Case: When you want class members to be accessible from outside the class.

```
#include <iostream>

using namespace std;

class MyClass {
public:
    int var; // Public variable

    void exampleMethod () { // Public method
        cout << "This is a public method." << endl;
    }
};

int main() {
    MyClass obj;
    obj.var = 10; // Directly accessible
    obj.exampleMethod(); // Directly accessible
    cout << "Public Variable: " << obj.var <<
endl;

    return 0;
}
```

Private

Definition: Members declared as private can only be accessed within the class itself. They are not accessible from outside the class.

Use Case: Private members are used when you want to restrict access to certain data or functionality within the class, keeping it hidden from outside interference.

By default all the members of a class would be private if access specifier is not explicitly stated.

Friend functions can also access private members.

```

#include <iostream>
using namespace std;
class MyClass {
private:
    int var; // Private variable
    void exampleMethod() { // Private method
        cout << "This is a private method." << endl;
    }
public:
    // Public method to access private members
    void setVar(int value) {
        var = value; // Accessing private variable
    }
    // Public method to access private method
    void callExampleMethod() {
        exampleMethod(); // Accessing private method
    }
    // Public method to display private variable
    void displayVar() {
        // Accessing private variable
        cout << "Private Variable: " << var << endl;
    }
};

```

```

int main() {
MyClass obj;
obj.var = 10; // Error: 'int
MyClass::var' is private within this
context
obj.exampleMethod(); // Error: 'void
MyClass::exampleMethod()' is private
within this context

obj.setVar(10); // Accessing private
variable using public method
obj.callExampleMethod(); // Accessing
private method using public method
obj.displayVar(); // Accessing
private variable using public method

return 0;
}

```

Protected

A protected member variable or function is very similar to a private member but it is provided one additional benefit that they can be accessed in child classes which are called derived classes. **[More on Inheritance Chapter]**

Summary:

public:

- Members are accessible from outside the class.

private:

- Members are accessible only within the class..

protected:

- Members are accessible in derived classes.

Member Functions

Definition:

- Functions defined within a class to manipulate data members.
- Member functions are used to perform operations on the data members of the class, which are the variables defined inside the class.

DEFINING MEMBER FUNCTION:

Member Function can be defined in two places:

- Outside the class
- Inside the class

Defining Member Functions Inside the Class

Function is defined inside the class.

Syntax:

```
class ClassName {  
public:  
    return_type functionName()  
{  
    // Function body  
}  
};
```

Example:

```
class Car {  
public:  
    void startEngine () {  
        cout << "Engine started." << endl;  
    }  
};
```

Defining Member Functions Outside the Class

Function is declared inside the class and defined outside using scope resolution operator (::).

Syntax:

```
class ClassName {  
public:  
    return_type functionName ();  
};  
  
return_type ClassName ::functionName ()  
{  
    // Function body  
}
```

Example:

```
class Car {  
public:  
    void startEngine ();  
};  
  
void Car::startEngine () {  
    cout << "Engine started." <<  
endl;  
}
```

Getters and Setters

Accessors (Getters):

- Functions that retrieve the value of data members.
- Provide read-only access to private data members.

Mutators (Setters):

- Functions that set or modify the value of data members.
- Provide a controlled way to modify private data members.

```
#include <iostream>
using namespace std;
class Car {
private:
    string brand;
    string model;
    int year;
public:
    // Function to set data values
    void setData(string b, string m, int y);
    // Function to retrieve data values
    void getData() {
        cout << "Brand: " << brand << ", Model: " << model << ",
        Year: " << year << endl; }
};

void Car::setData(string b, string m, int y) {
    brand = b;
    model = m;
    year = y; }
```

```
int main() {
    Car myCar;
    myCar.setData("Toyota", "Corolla",
2020);
    myCar.getData();
    return 0;
}
```

Try it!!

Q2. Write a program to create a class named **Rectangle** that has private member variables for **length** and **breadth**. Provide public member functions to set the dimensions, calculate the area, and calculate the perimeter of the rectangle. Demonstrate the functionality by creating an object of the Rectangle class and using it to calculate and display the area and perimeter.

Private Member Functions

- Some situations may require functions to be hidden from the outside calls. We can place these functions in the private section.
- Private member functions are functions declared within a class that can only be accessed by other member functions of the same class.
- Even an object can not invoke a private function using the dot operator.

Purpose:

- Encapsulate internal logic.
- Hide implementation details.
- Ensure data integrity.

```
#include <iostream>
using namespace std;

class Circle {
private:
    // Private variable
    float radius;
    // Private member function
    void Area() {
        cout << "Area: " << 3.14*radius*radius << endl;
    }
public:
    // Function to set radius value
    void setRadius(float r) {
        radius = r;
    }
    // Function to call private function: Area()
    void getArea() {
        Area();
    }
};
```

```
int main() {
    float radius;
    cout << "Enter Radius: " <<
endl;
    cin >> radius;
    // create object
    Circle c;
    // call set radius
    c.setRadius(radius);
    // call get area function
    c.getArea();
    return 0;
}
```

Static Data Members

- Static data members are variables of a class that are shared among all objects of the class.
- They are declared using the **static** keyword.
- Unlike regular data members, there is only one copy of a static data member, regardless of how many objects of the class are created.
- Static data members belong to the class itself.
- Static members exist even if no objects of the class have been instantiated.
- They are created when the program starts, and destroyed when the program ends.

```
class ClassName {  
public:  
    static data_type variable_name; // Declaration  
};
```

Characteristics of Static Data Members

- **Shared Among All Objects:** Only one instance exists, shared by all instances of the class.
- **Class-Level Scope:** Can be accessed without creating an instance of the class.
- **Initialization:** Must be defined and initialized outside the class definition.
- **Access:** Can be accessed using the class name or through objects of the class.

Initialization of Static Data Members

- **Default Initialization:** Static data members are zero-initialized by default when the first object of the class is created.
- **Explicit Initialization:** It's a good practice to explicitly initialize static data members outside the class definition.

```
data_type ClassName::variable_name = 0;
```

Example of Default Initialization

```
#include <iostream>
using namespace std;

class Counter {
public:
    static int count; // Static data member declaration
};

// No explicit initialization, it will be zero-initialized by default
int Counter::count;

int main() {
    Counter obj1;
    cout << "Counter::count (before increment): " << Counter::count << endl; // outputs: 0
    Counter::count++; // Increment the static member
    cout << "Counter::count (after increment): " << Counter::count << endl; // outputs: 1

    cout << "Count value using object: " << obj1.count << endl; // outputs: 1
    return 0;
}
```

Example of Explicit Initialization

```
#include <iostream>
using namespace std;
class Counter {
public:
    static int count; // Static data member declaration
};
// Explicit initialization
int Counter::count = 10;
int main() {
    Counter obj1;
    cout << "Counter::count (before increment): " << Counter::count << endl; // outputs: 10
    Counter::count++; // Increment the static member
    cout << "Counter::count (after increment): " << Counter::count << endl; // outputs: 11

    cout<< "Count value using object: " << obj1.count<< endl; // outputs: 11
    return 0;
}
```

Static Member Functions

- Like static member variables, static member functions are not attached to any particular object.
- They can directly access other static members (variables or functions), but not non-static members.
- They can be called directly by using the class name and the scope resolution operator.

```
ClassName::staticMemberFunctionName (params) ;
```

```
#include <iostream>

using namespace std;

class Car {
private:
    static int count; // Static data member
public:
    // Static member function to get thecount
    static int getcount() {
        return count;
    }
    // Static member function to increment the
    count
    static void incrementcount() {
        count++;
    }
};
```

```
// Definition of the static data member
int Car::count = 0;

int main() {
    // Preferred way: Using the class name
    Car::incrementcount();
    Car::incrementcount();

    // Also possible: Using an object
    Car myCar;
    myCar.incrementcount();

    cout << "Number of Car objects: " <<
    Car::getcount() << endl; // Output: 3

    return 0;
}
```

Try it!!

Q3. Write a program to create a class named **Student** that includes **private** member variables for the student's **name** and **roll** number. It should also include member functions, a static data variable to keep track of the number of students created, and a static member function to retrieve this count. Demonstrate the functionality by creating objects of the **Student** class and using its member functions.



Any Queries?